

SESSION 1 / 4

Agent concepts & architectures

What is an intelligent agent? Taxonomy, action loop and the PEAS framework

Guillaume Hochard · Airpanel | 22–24 June 2026

The module in 4 sessions

Session	Topic	Lab / project
1	Agent concepts & architectures	Lab 1 — first tool
2	From LLM to agent: building an agent	Lab 2 — ReAct
3	Industrial frameworks & multi-agents	Lab 3 — LangGraph/CrewAI
4	Production, safety & Craft AI project	Craft AI project

01

What is an agent?

Definitions, rationality and limits



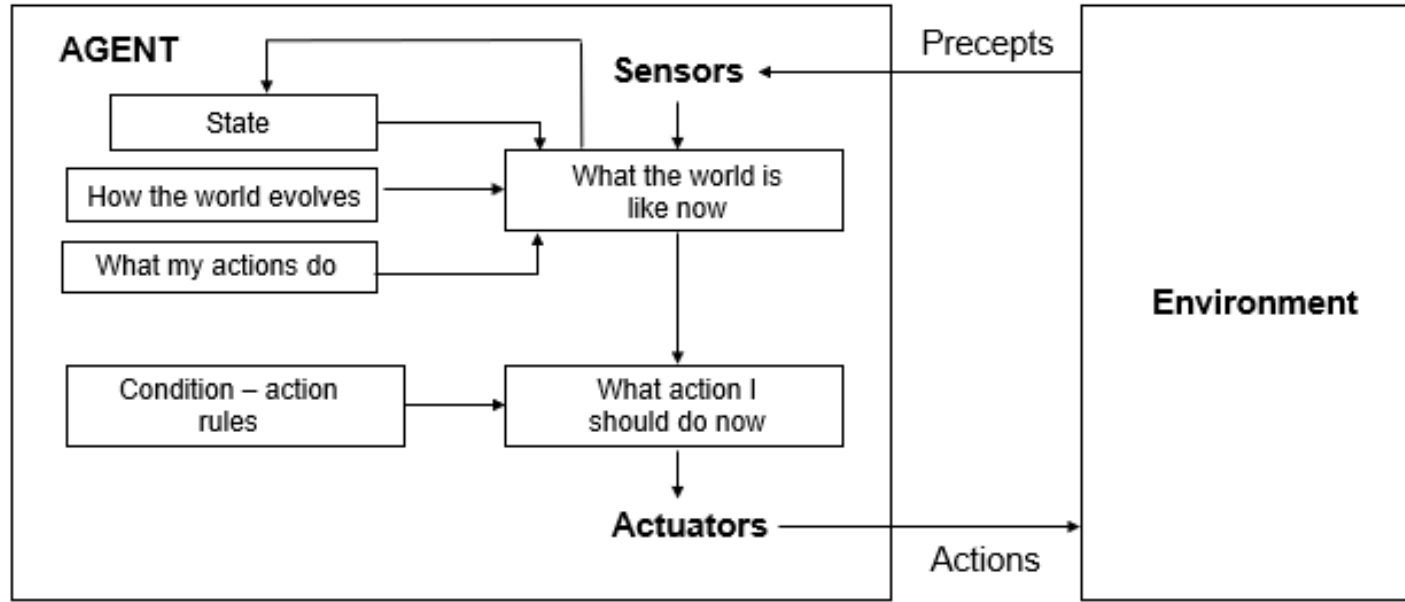
Definition & rationality

An agent perceives its environment and acts on it to achieve a goal.

- Sensors (perception) → agent function → actuators (action).
- Autonomy: the agent chooses its actions; they are not all scripted.
- Rational agent: maximizes its expected performance measure, given what it perceives.
 - rationality ≠ omniscience: it decides with partial information and limited resources (bounded rationality).
- Key difference from a classic program: the decision depends on the perceived state and a goal, not on a fixed path.

Definition & rationality

An agent perceives its environment and acts on it to achieve a goal.



Three levels not to confuse

- Classic program: input → deterministic output, no notion of a goal.
- LLM alone: generates plausible text in one shot, without acting or verifying.
- Agent (LLM + tools + loop): reasons, acts on the world, observes, repeats.
- It is the addition of tools AND a decision loop that turns the model into an agent.

Taxonomy of agents

Reactive (simple reflex)

Condition-action rules on the current perception, without memory.

E.g. thermostat, basic NPC

Deliberative (model-based)

Maintains an internal state and a world model to decide.

E.g. mapping robot vacuum

Goal-oriented

Reasons about goals and plans a sequence of actions.

E.g. route planner

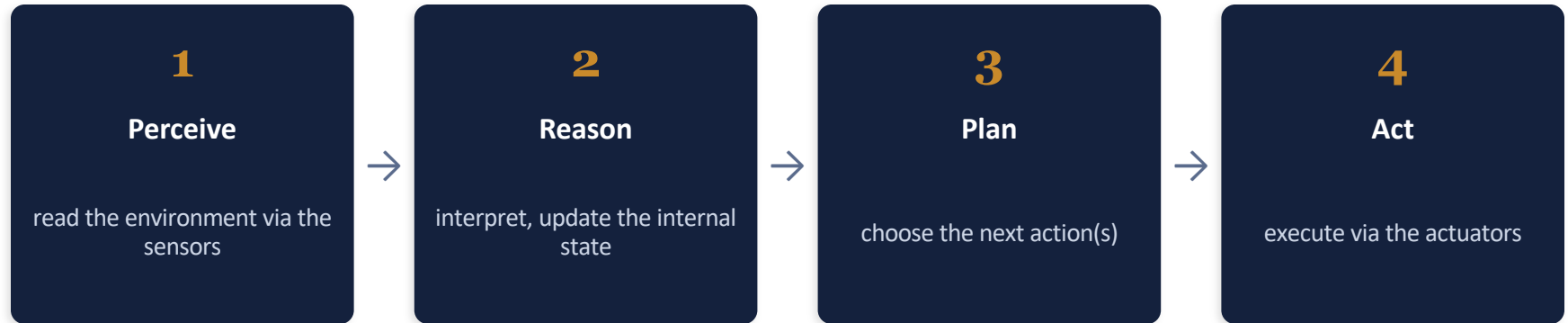
Learning

Improves its behavior with experience.

E.g. RL agent, recommendation

The agent loop

Fundamental cycle, repeated at each time step:



PEAS framework — specify before coding

PEAS = an agent's identity card, to fill in BEFORE writing a single line of code.

Dimension	Question	Example: PGE5 revision assistant
Performance	How do we measure success?	Accuracy, relevance, student satisfaction
Environment	Where does the agent operate?	Courses, questions, syllabus documents
Actuators	What actions are possible?	Answer, search, generate a quiz
Sensors	What inputs are perceived?	Question text, course files

Example — PGE5 revision assistant: P = correct quiz answers · E = course corpus + student history · A = generate quizzes, plan · S = questions, past scores.

Characterizing an environment

Observable / partially

Does the agent see the whole state, or only part of it?

E.g. poker = partially observable

Deterministic / stochastic

Is the effect of an action certain?

E.g. an API that sometimes fails = stochastic

Episodic / sequential

Are decisions independent or chained together?

E.g. dialogue = sequential

Static / dynamic

Does the environment change while the agent is thinking?

E.g. trading = highly dynamic

An agent in a few lines

The perceive → reason → act loop, in Python — exactly what you will code in Labs 1–2.

```
# 1) Describe a tool; the model DECIDES to call it
registry.register(time_tool, get_current_time)
messages = [{"role": "user", "content": "What time is it?"}]

while True:
    reply = client.complete(messages, tools=registry.specs)
    messages.append(reply.to_message())
    if not reply.has_tool_calls:      # no more tools → final answer
        break
    for call in reply.tool_calls:     # OUR code executes the tool
        obs = registry.call(call["name"], call["arguments"])
        messages.append({"role": "tool", "content": obs}) # observation
```

The model chooses the action; the code executes it and returns the observation — then we loop again.

Lab 1 — Setup & first tool call

Notebook: `lab_01_setup_et_premier_outil.ipynb`

- Set up the Python environment and the provider-agnostic LLM layer (.env, API key).
- Declare a first tool (JSON schema) and observe that the model decides to call it.
- Chain tool call → execution → final answer with `run_agent`.
- Formalize a use case with the PEAS grid.

Key takeaways



- An agent = perception → reasoning → planning → action, in a loop.
- Four families: reactive, deliberative, goal-oriented, learning — often combined.
- PEAS and the environment properties frame any design.
- What makes the agent: tools + a decision loop around the LLM.
- Next step: concretely turn an LLM into an agent.